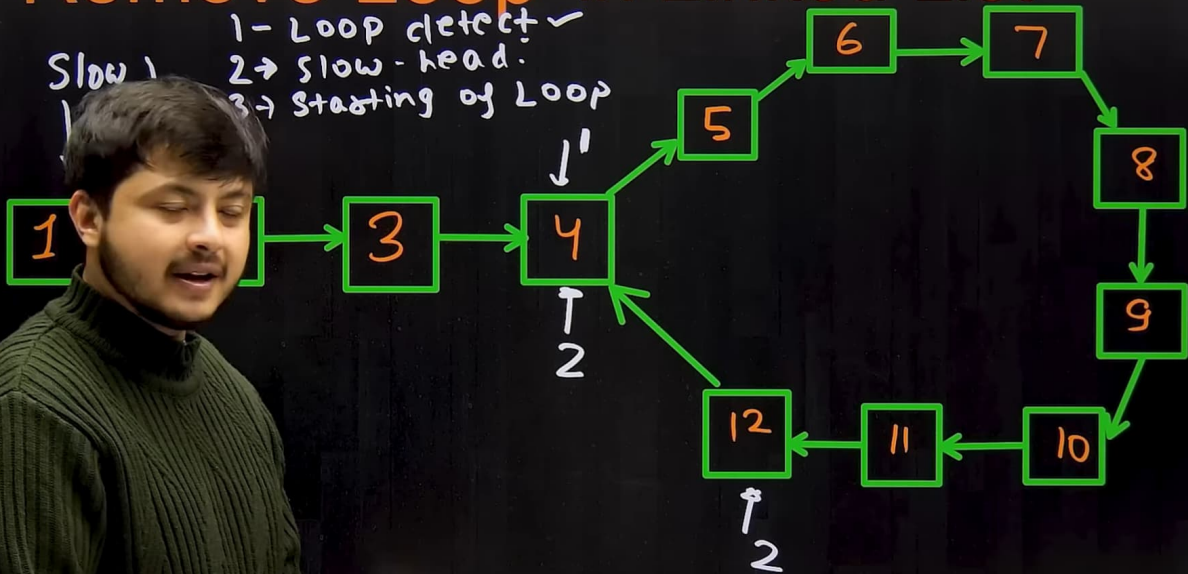


Remove Loop in Linked List

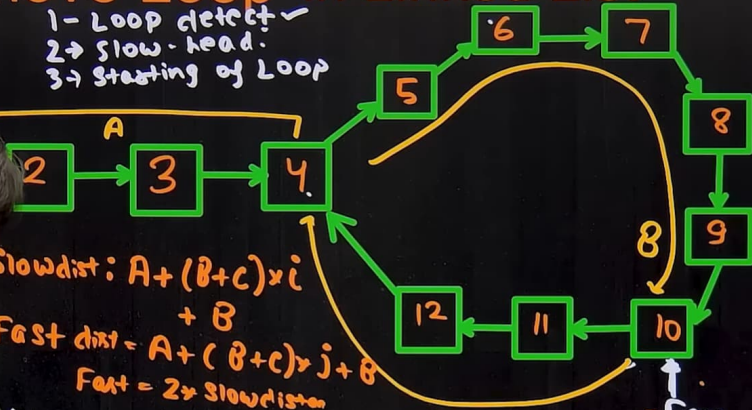
- 1 - Loop detect ✓
2 → slow - head.
3 → starting of Loop



Remove Loop in Linked List

- 1- Loop detect ✓
- 2 → slow - head.
- 3 → starting of Loop

Slow ↑



$$\text{Slow dist: } A + (B+C) \times i$$

$$+ B$$

$$\text{Fast dist} = A + (B+C) \times j + B$$

$$\text{Fast} = 2 \times \text{Slow dist}$$

$$(B+C) \times j + B = 2A + 2(B+C) \times i + 2B$$

$$A + B = (B+C) \times j - 2(B+C) \times i$$

$$A + B = (B+C) \times (j - 2i)$$

$$A + B = T \times (B+C)$$

$$A + B = B + C$$

$$\underline{A = C}$$

j = Fast
walene
Kitne
chakker
lagayn

i = Slow, loop

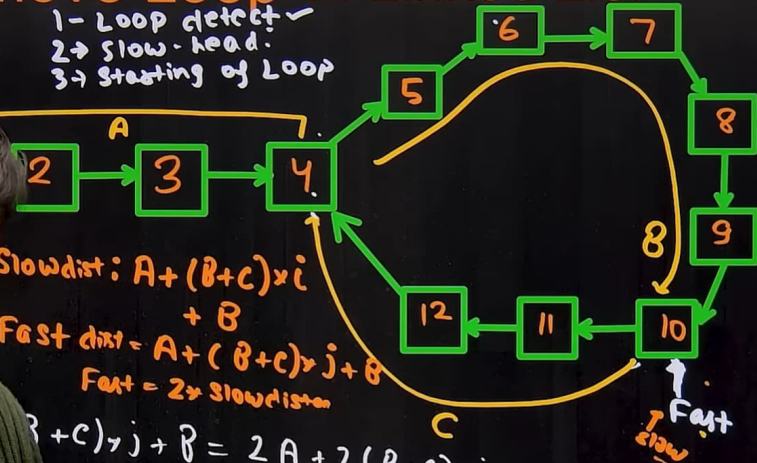
$$\boxed{j > 2i}$$

Fast
Slow

Remove Loop in Linked List

- 1- Loop detect ✓
 2 → slow - head.
 3 → starting of Loop

Slow ↑



$$\text{Slow dist: } A + (B+C) \times i$$

$$\text{Fast dist: } A + (B+C) \times j + B$$

$$\text{Fast} = 2 \times \text{Slow dist}$$

$$(B+C) \times j + B = 2A + 2(B+C) \times i + 2B$$

$$A + B = (B+C) \times j - 2(B+C) \times i$$

$$A + B = (B+C) \times (j - 2i)$$

$$A + B = T \times (B+C)$$

$$A + B = 2B + 2C$$

$$A = (B+C) + C$$

j = Fast
 walene
 Kitne
 Chakke
 lagayn

i = Slow, loop

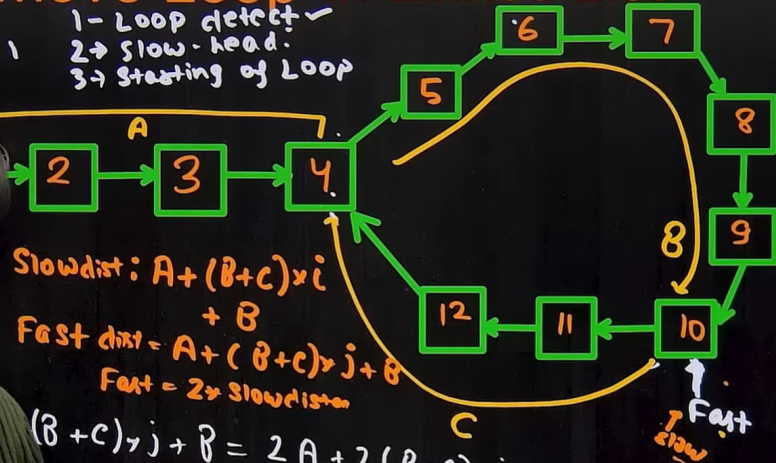
$$j > 2i$$

Fast
 Slow

Remove Loop in Linked List

- 1- Loop detect ✓
- 2 → slow-head.
- 3 → starting of Loop

Slow



$$\text{Slow dist: } A + (B+C) \times i$$

$$\text{Fast dist: } A + (B+C) \times j + B$$

$$\text{Fast} = 2 \times \text{Slow dist}$$

$$(B+C) \times j + B = 2A + 2(B+C) \times i + 2B$$

$$A + B = (B+C) \times j - 2(B+C) \times i$$

$$A + B = (B+C) \times (j - 2i)$$

$$A + B = T \times (B+C)$$

$$A + B = 3B + 3C$$

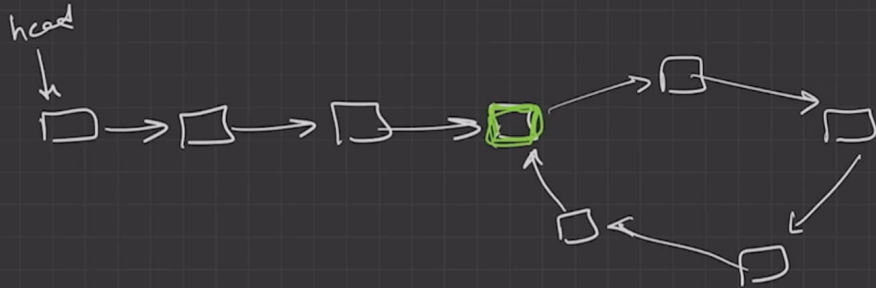
$$A = 2(B+C) + C$$

j = Fast
walene
Kitne
chakker
lagayn

i = Slow, loop

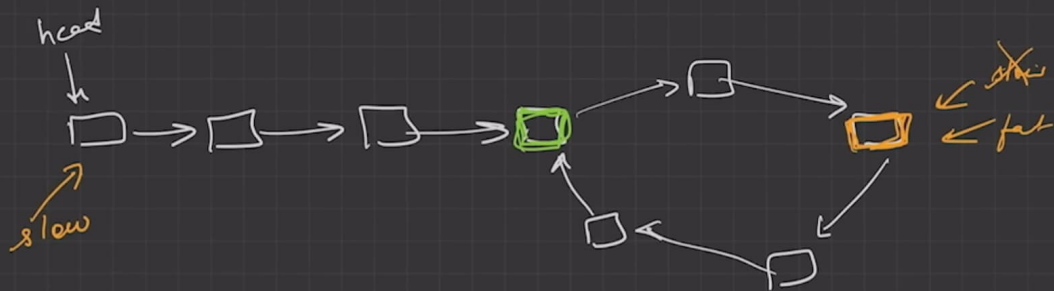
$$j > 2i$$

Fast
Slow



Ques:- Starting Node of Loop





Ques:- Starting Node of Loop

Approach:-

I $\rightarrow$ FCD Algo $\rightarrow$ Point of Intersection

II $\rightarrow$ slow = head



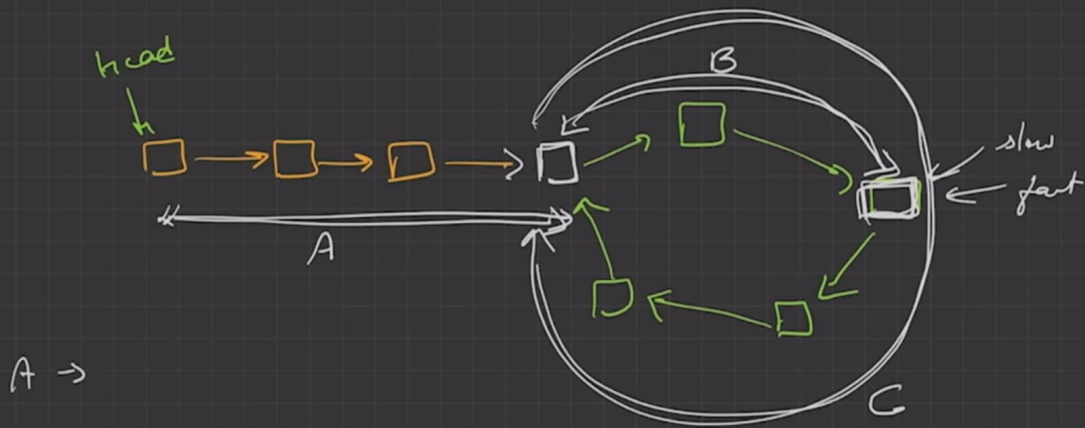
II $\rightarrow$ slow = head $\rightarrow$ slow, fast $\rightarrow$ same place

when (slow == fast)

$\rightarrow$ starting point of loop



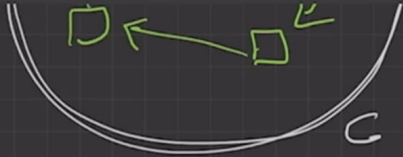
starting point of Loop



A →



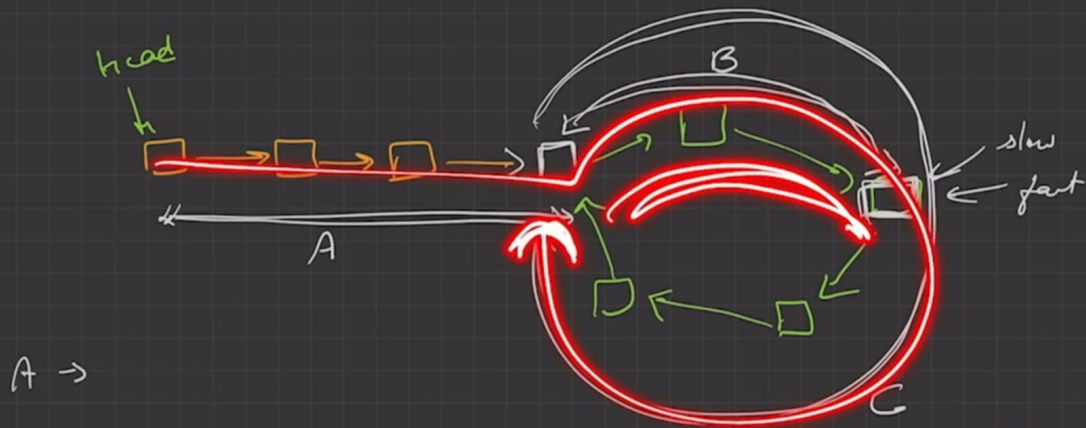
A →



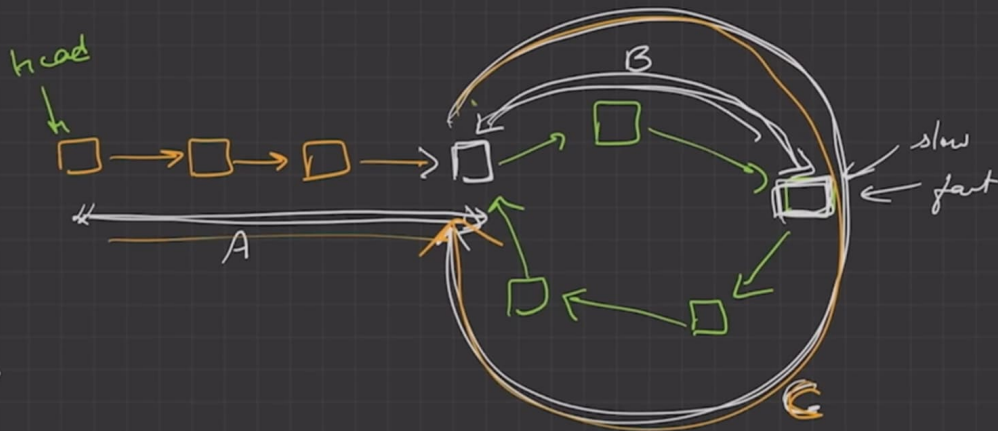
Distance by fast pointer = 2 ★ Distance by slow pointer



starting point of Loop



A →



Distance by slow = 2 * Distance by fast



Distance by
fast pointer = 2 * Distance by
slow pointer

$$(A + x * C + B) = 2 * (A + y * C + B)$$

$$A + xC + B = 2A + 2yC + 2B$$

$$\frac{C(x - 2y)}{K} = A + B$$



$$A + xC + B = 2A + 2yC + 2B$$

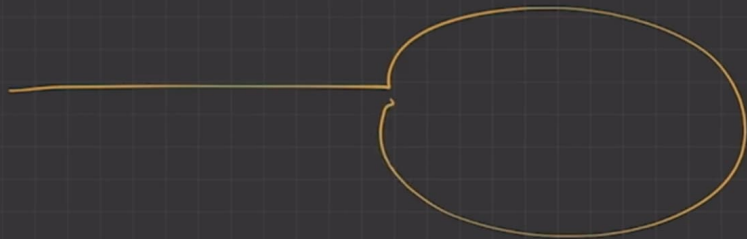
$$\underbrace{C(x-2y)}_K = A + B$$

$$A + B = K \text{ times } C$$

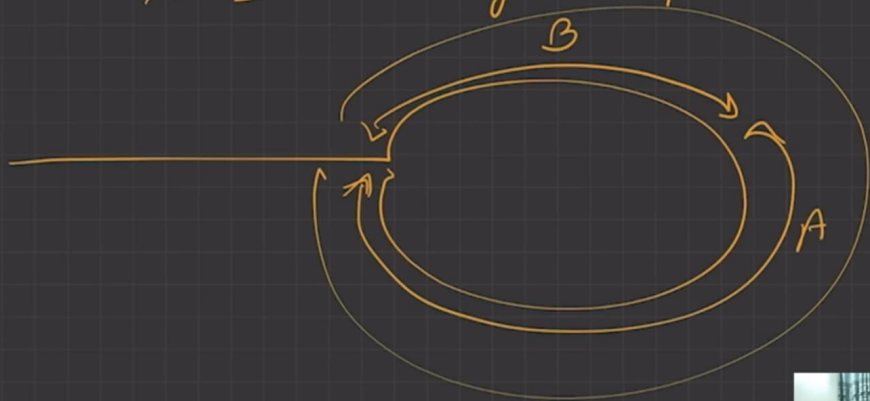


$$\overset{K}{\boxed{A + B = \underline{\underline{K \text{ times } C}}}}$$

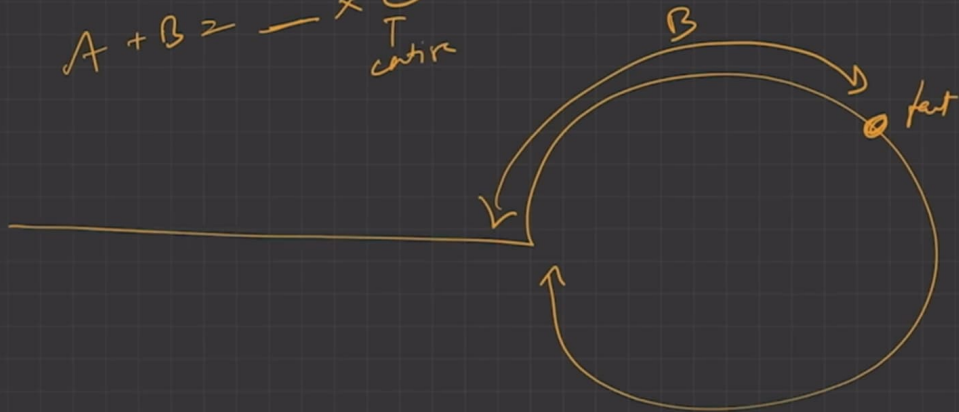
$\Rightarrow A + B \rightarrow \text{cycle complete Kardi}$

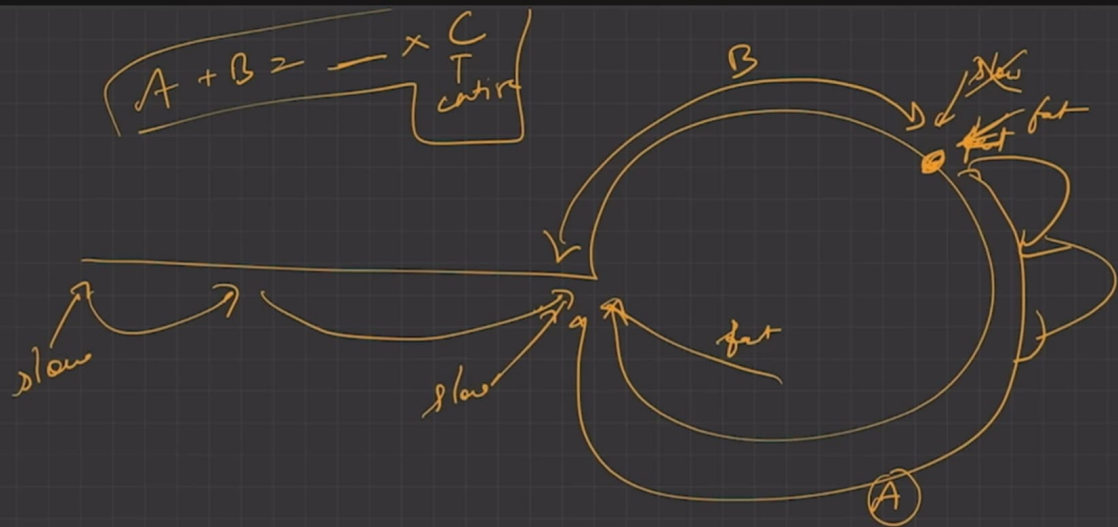


$\Rightarrow A + B \rightarrow$ Cycle complete Kardi



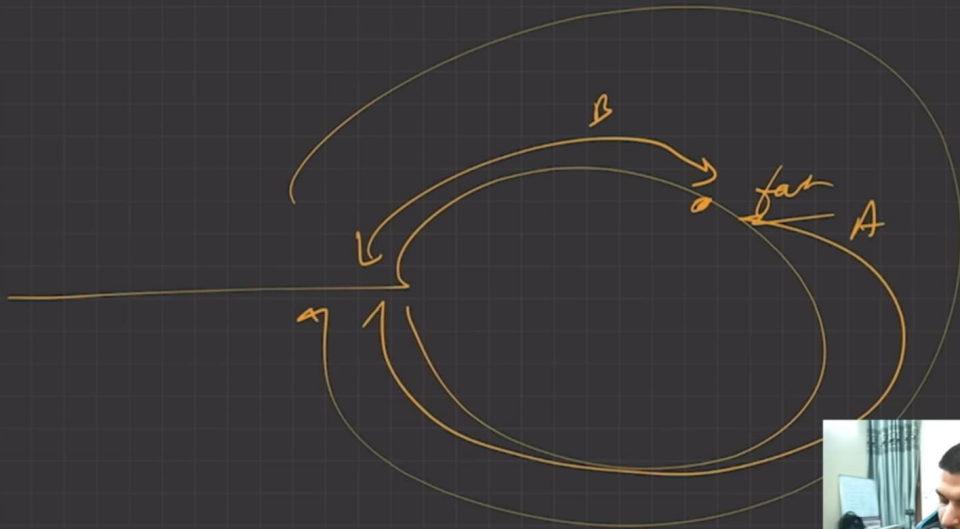
$$A + B = - \times \frac{C}{T_{\text{active}}}$$





$$A + B \geq \star C$$

entire cycle



```
while (slow->next  
      != fast  
      {  
    slow = slow->next;
```

```
slow->next  
= NULL;
```

Node *slow = head, *fast = head;

```
while (fast != NULL && fast->next != NULL)  
{  
    slow = slow->next;  
    fast = fast->next->next;  
    if (slow == fast)  
        break;  
}
```

```
if (fast == NULL || fast->next == NULL)  
    return;
```

```
slow = head
```

```
while (slow != fast)  
{  
    slow = slow->next;  
    fast = fast->next;  
}
```

Node *slow = head, *Fast = head;

while (Fast != NULL && Fast->next != NULL)

{

slow = slow->next;

Fast = Fast->next->next;

if (slow == Fast)

break;

}

prev->next = NULL;

if (Fast == NULL || Fast->next == NULL)
return;

slow = head

Node * prev = NULL;

while (slow != Fast)

{

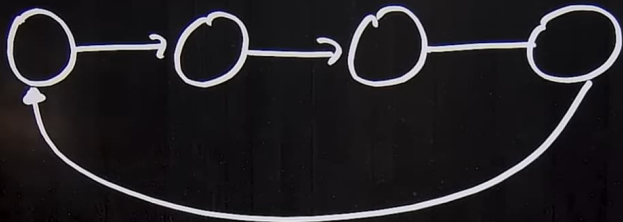
slow = slow->next;

Fast = Fast->next;

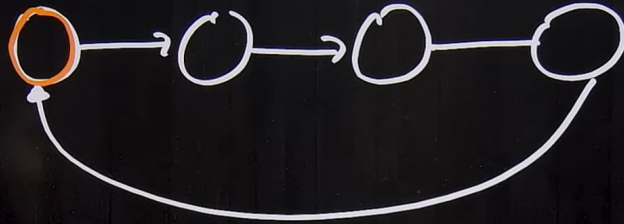
prev = Fast;

}

↓ ↓
1 2



2!



$prev = NULL;$

$prev \rightarrow next = NULL;$

Node *slow = head, *fast = head;

while (fast != NULL && fast->next != NULL)

{

slow = slow->next;

fast = fast->next->next;

if (slow == fast)

break;

fast = NULL;

}

if (fast == NULL || fast->next == NULL)
return;

slow = head

Node * prev = ~~NULL~~ ^{fast};

while (slow != fast)

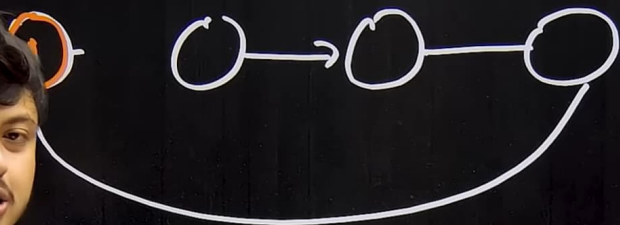
{

slow = slow->next;

fast = fast->next;

prev = fast;
}

2!



$prev = \text{NULL};$

$prev \rightarrow next = \text{NULL};$
 $1 \rightarrow next$



Remove loop in Linked List



Medium Accuracy: 27.66% Submissions: 422K+ Points: 4

30+ People have Claimed their 90% Refunds. Start Your Journey Today!

Given a linked list of N nodes such that it may contain a loop.

A loop here means that the last node of the link list is connected to the node at position X(1-based index). If the link list does not have any loop, X=0.

Remove the loop from the linked list, if it is present, i.e. unlink the last node which is forming the loop.

Example 1:

Input:

N = 3

value[] = {1,3,4}

X = 2

Output: 1

Explanation: The link list looks like

1 -> 3 -> 4

^ |

|

```
90 public:
91 //Function to remove a loop in the linked list.
92 void removeLoop(Node* head)
93 {
94     // code here
95
96     // Detect the loop
97
98     Node* slow = head, *fast = head;
99
100     while(fast && fast->next)
101     {
102         slow = slow->next;
103         fast = fast->next->next;
104
105         if(slow == fast)
106             break;
107     };
108
109     if(fast == NULL || fast->next == NULL)
110         return;
```





</> Problem

Editorial

Submissions

Comments

C++ (g++ 5.4)

Average Time: 45m

Start Timer



Output Window

Compilation Results

Custom Input

[Suggest Feedback](#)

Problem Solved Successfully ✓

💡 You get marks only for the first correct submission if you solve the problem without viewing the full solution.

Test Cases Passed:

151 / 151

Your Total Score:

2142

Total Time Taken:

0.95

Correct Submission

Count:

6

Attempts No.:

13

Next Suggested Problem(s):

Rotate a Linked List

Delete nodes having greater value on right

Find the first node of loop in linked list

```
109 if(fast==NULL || fast->next==NULL)
110 return;
111
112 slow = head;
113 // Meet karwao
114 while(slow!=fast)
115 {
116     slow = slow->next;
117     fast = fast->next;
118 }
119
120
121 // slow last node pe
122 while(slow->next!=fast)
123 {
124     slow = slow->next;
125 }
126
127 slow->next= NULL;
128
129
130
```



Custom Input

Compile & Run

Submit

{Three
90
Challenger

Remove Loop in Linked List

1 → Loop detect

Slow

Fast

1 2 Slow Fast

In this video



Chapters

Transcript



Method 2 :
Remove Loop in
Linked List

35:29



Code Part - Method 2 -
Remove Loop in Linked
List

41:57

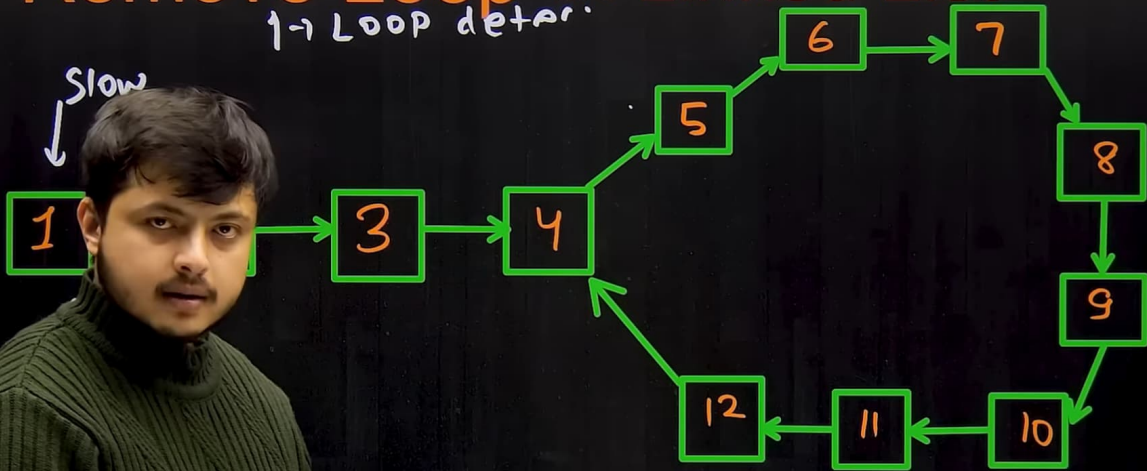


Problem 2 -
Intersection Point in Y
Shaped Linked List

47:38

Remove Loop in Linked List

1- Loop detect

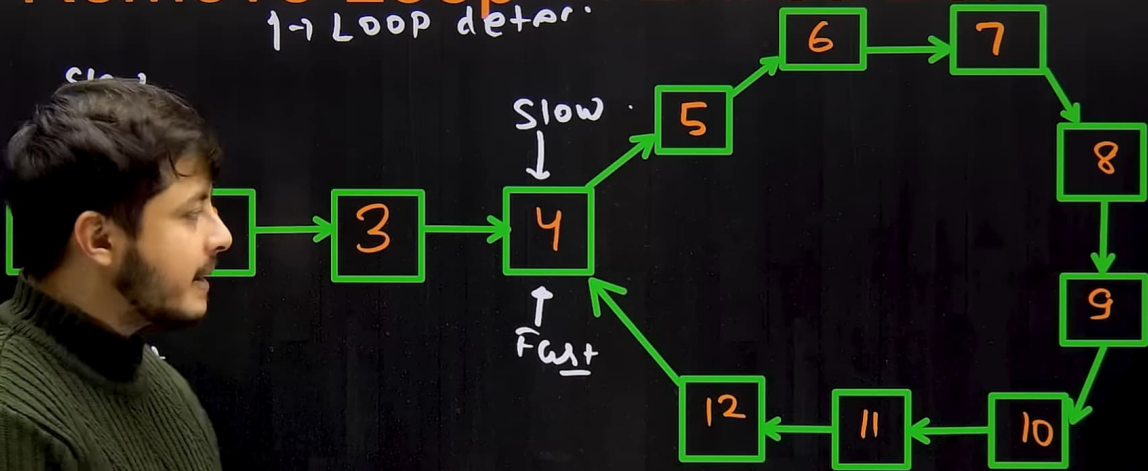


9- Length

1 2
↑ ↑
Slow Fast

Remove Loop in Linked List

1- Loop detar.

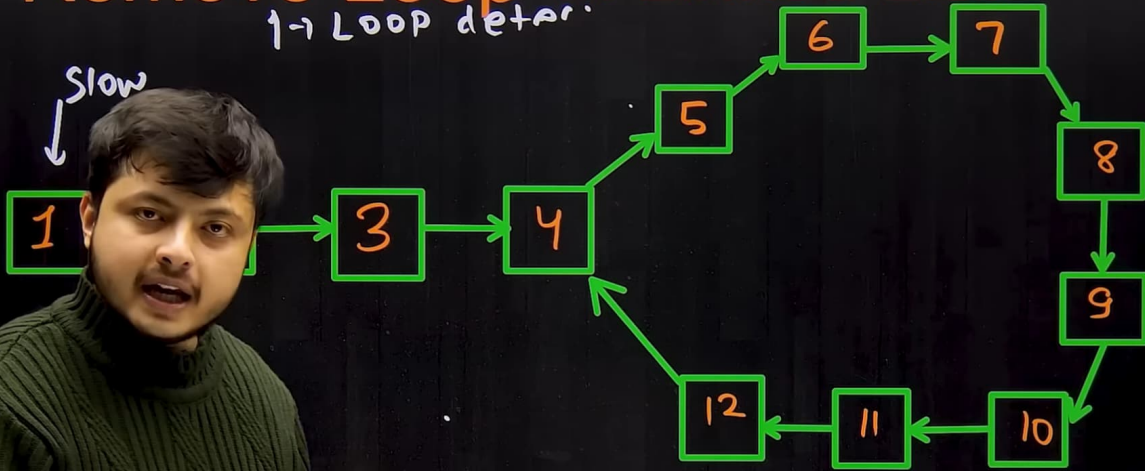


9- Length

1 2
↑ ↑
Slow Fast

Remove Loop in Linked List

1- Loop detect.



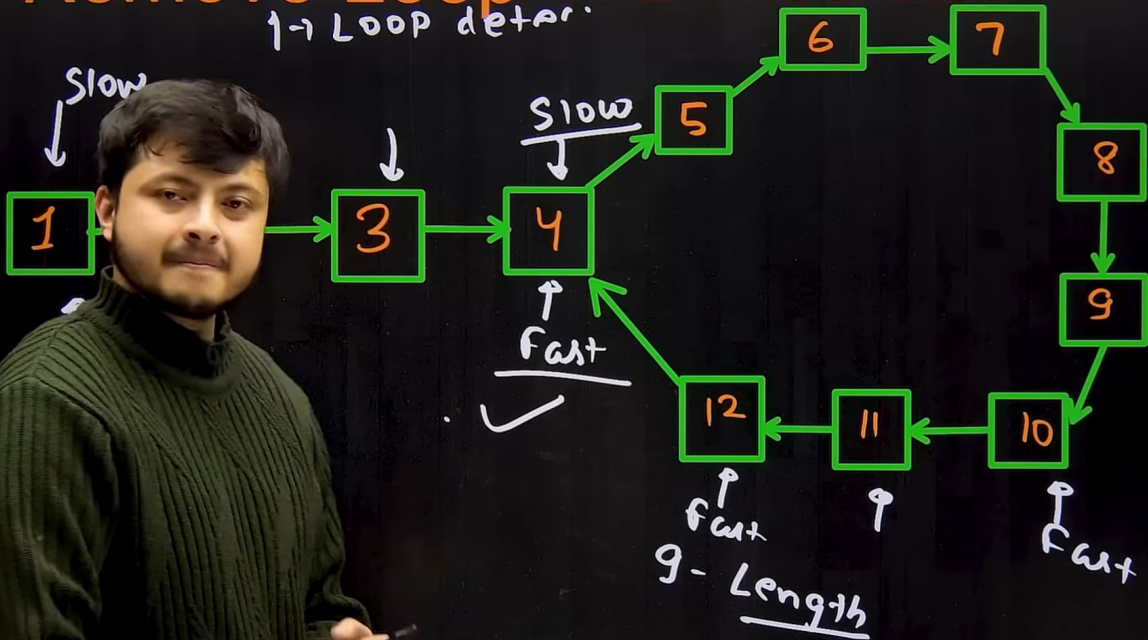
आ जाएगा अभी भी दोनों के बीच की डिस्टेंस कितनी है नौ ही है

9 - Length

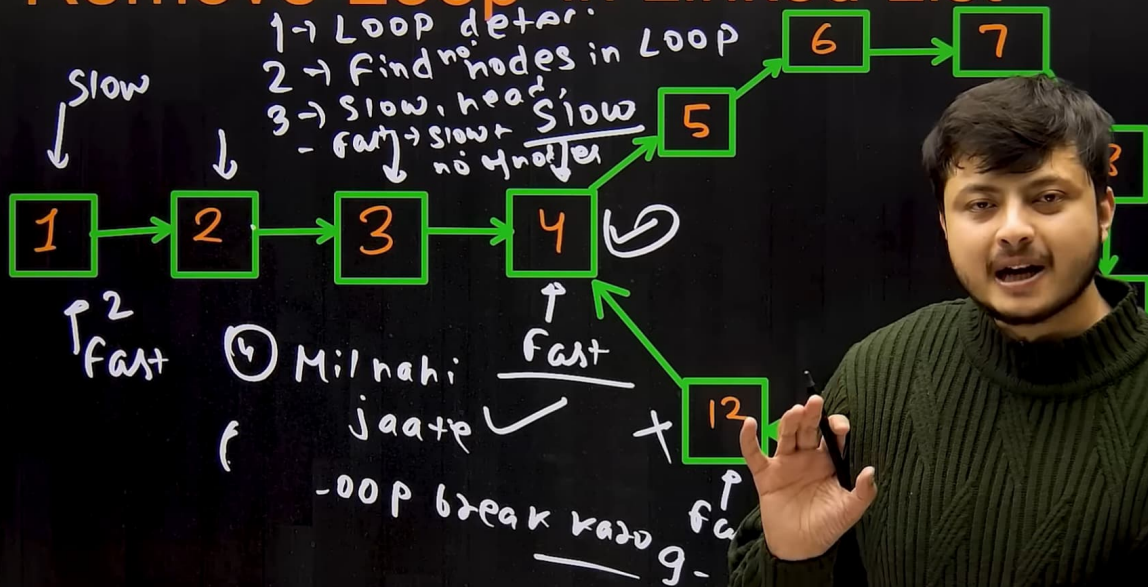
Fast

Remove Loop in Linked List

1- Loop detar.



Remove Loop in Linked List





</> Problem

Editorial

Submissions

Comments

C++ (g++ 5.4)

Average Time: 45m

Start Timer



Remove loop in Linked List



Medium

Accuracy: 27.66%

Submissions: 422K+

Points: 4

30+ People have Claimed their 90% Refunds. Start Your Journey Today!

Given a linked list of N nodes such that it may contain a loop.

A loop here means that the last node of the link list is connected to the node at position X(1-based index). If the link list does not have any loop, X=0.

Remove the loop from the linked list, if it is present, i.e. unlink the last node which is forming the loop.

Example 1:

Input:

N = 3

value[] = {1,3,4}

X = 2

Output: 1

Explanation: The link list looks like

1 -> 3 -> 4

^ |

|

95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115

```
// CODE HERE

// Just detect the loop

Node *slow = head, *fast = head;

while(fast&&fast->next)
{
    slow = slow->next;
    fast = fast->next->next;

    if(slow==fast)
        break;
}

//
if(!fast||!fast->next)
return;
```



Custom Input

Compile & Run

Submit

{Three
90
Challen



</> Problem

Editorial

Submissions

Comments

C++ (g++ 5.4)

Average Time: 45m

Start Timer



Remove loop in Linked List



Medium

Accuracy: 27.66%

Submissions: 422K+

Points: 4

30+ People have Claimed their 90% Refunds. Start Your Journey Today!



Given a linked list of N nodes such that it may contain a loop.

A loop here means that the last node of the link list is connected to the node at position X(1-based index). If the link list does not have any loop, X=0.

Remove the loop from the linked list, if it is present, i.e. unlink the last node which is forming the loop.

Example 1:

Input:

N = 3

value[] = {1,3,4}

X = 2

Output: 1

Explanation: The link list looks like

1 -> 3 -> 4

^ |

|

```
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
```

```
if(!fast || fast->next)
return;

// Find the length of loop

slow = fast->next;
int count = 1;

while(slow!=fast)
{
    count++;
    slow=slow->next;
}

// Loop ke andar kitne nodes hai;
```

slow = head, fast = head;



Custom Input

Compile & Run

Submit

{Three
90
Challen



</> Problem

Editorial

Submissions

Comments

C++ (g++ 5.4)

Average Time: 45m

Start Timer



Remove loop in Linked List



Medium

Accuracy: 27.66%

Submissions: 422K+

Points: 4

30+ People have Claimed their 90% Refunds. Start Your Journey Today!



Given a linked list of N nodes such that it may contain a loop.

A loop here means that the last node of the link list is connected to the node at position X(1-based index). If the link list does not have any loop, X=0.

Remove the loop from the linked list, if it is present, i.e. unlink the last node which is forming the loop.

Example 1:

Input:

N = 3

value[] = {1,3,4}

X = 2

Output: 1

Explanation: The link list looks like

1 -> 3 -> 4

^ |

|

115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136

```
slow = fast->next;  
int count = 1;
```

```
while(slow!=fast)  
{  
    count++;  
    slow=slow->next;  
}
```

// Loop ke andar kitne nodes hai;

```
slow = head, fast = head;
```

// Fast ko kitne disrtance door rkhna hai, jo h

```
while(count--)  
{  
    fast = fast->next;  
};|
```



Custom Input

Compile & Run

Submit

{Three
90
Challen



</> Problem

Editorial

Submissions

Comments

C++ (g++ 5.4)

Average Time: 45m

Start Timer



Remove loop in Linked List



Medium

Accuracy: 27.66%

Submissions: 422K+

Points: 4

30+ People have Claimed their 90% Refunds. Start Your Journey Today!



Given a linked list of N nodes such that it may contain a loop.

A loop here means that the last node of the link list is connected to the node at position X(1-based index). If the link list does not have any loop, X=0.

Remove the loop from the linked list, if it is present, i.e. unlink the last node which is forming the loop.

Example 1:

Input:

N = 3

value[] = {1,3,4}

X = 2

Output: 1

Explanation: The link list looks like

1 -> 3 -> 4

^ |

|

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

slow = head, fast = head;

// Fast ko kitne distance door rkhna hai, jo h

while(count--)

{

fast = fast->next;

};

while(slow!=head)

{

slow = slow->next;

fast = fast->next;

};

// Bring slow to last node

while(slow->next!=fast)

slow = slow->next;



Custom Input

Compile & Run

Submit

{Three
90
Challen



Output Window

Compilation Results Custom Input

[Suggest Feedback](#)

Problem Solved Successfully ✓

💡 Calculating score...

Test Cases Passed:

151 / 151

Total Points Scored:



Total Time Taken:

1.38

Your Accuracy:



Attempts No.:



Suggested Contest:

Based on your excellent performance, we believe you are fully prepared to participate in this upcoming contest.



GFG Weekly Coding Contest

Participate every Sunday at 7 PM. Practice DSA, compete with peers, win Geekbits!

[Participate now →](#)

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

slow = head, fast = head;

// Fast ko kitne distance door rkhna hai, jo h

while(count--)

{

fast = fast->next;

};

while(slow!=fast)

{

slow = slow->next;

fast = fast->next;

};

// Bring slow to last node

while(slow->next!=fast)

slow = slow->next;

slow->next = NULL;



[Custom Input](#)

Compile & Run

Submit

{Three
90
Chall